

Naive Bayes [Part-1]

Problem in Bayes Theorem:-

The problem comes when we cannot compute or estimate probabilities accurately.

In real datasets, the joint probability of many features together is extremely hard to calculate.

As the no. of features increase data becomes sparse (curse of dimensionality)

$$P(X_1, X_2, X_3, \dots | Y)$$

Because of this, Bayes theorem becomes impractical for high dimensional data.

Why Naive Bayes introduced:-

Naive Bayes simplifies Bayes Theorem to make it usable.

It assumes that all features are independent given the class. This converts a hard joint probability into a product of simple probabilities.

Instead of $P(X_1, X_2, X_3, \dots | Y)$

We compute
 $P(X_1 | Y) \cdot P(X_2 | Y) \cdot P(X_3 | Y)$

This assumption drastically reduces data requirements.

It makes the algorithm fast, scalable, and easy to train.

Intuition:-

outlook	Temp	Humidity	Windy	Play Tennis
Sunny	Hot	High	F	No
Sunny	Hot	High	T	No
Overcast	Hot	High	F	Yes
Rainy	Mild	High	F	Yes
Rainy	Cool	Normal	F	Y
Overcast	Cool	Normal	T	N
Sunny	Cool	Normal	T	Y
Sunny	Mild	High	F	N
Rainy	Cool	Normal	F	Y
Sunny	Mild	Normal	F	Y
Overcast	Mild	Normal	T	Y
Overcast	Mild	High	T	Y
Rainy	Hot	Normal	F	Y
Rainy	Mild	High	T	N

Bayes Theorem:- $P(A|B) = \frac{P(B|A) P(A)}{P(B)}$

given new condition:- { Sunny, Cool, Normal, True } = W

↓ probability

Yes No

$$P(\text{Yes}|W) = \frac{P(W|\text{Yes}) P(\text{Yes})}{P(W)} \Rightarrow \cancel{P(W|\text{Yes})} (9/14) \Rightarrow 0//$$

$$P(\text{No}|W) = \frac{P(W|\text{No}) P(\text{No})}{P(W)} \Rightarrow \cancel{P(W|\text{No})} (5/14) \Rightarrow 0//$$

$$P(\text{Sunny} \cap \text{Cool} \cap \text{Normal} \cap \text{True} | \text{Yes}) \Rightarrow 0//$$

$$P(\text{Sunny} \cap \text{Cool} \cap \text{Normal} \cap \text{True} | \text{No}) \Rightarrow 0//$$

⇒ not possible to compute/find the combination when the dataset has large no. of features.

$$P(\text{Sunny}|\text{Yes}) P(\text{Cool}|\text{Yes}) P(\text{Normal}|\text{Yes}) P(\text{True}|\text{Yes})$$

$$\frac{2}{9} \times \frac{3}{9} \times \frac{6}{9} \times \frac{3}{9} \times \frac{9}{14}$$

⇒ assumes all are independent, convert hard joint probability into the product of simple probabilities.

Mathematical Formulation:-

$$\underbrace{X_1, X_2, X_3, \dots}_{i/p} \quad \underbrace{Y}_{o/p}$$

for multiclassification

$$Y_1, Y_2, Y_3, \dots, Y_K$$

1, 2, 3, ..., K

classes

$$P(Y_1|X_T) = P(X_T|Y_1) P(Y_1)$$

$$P(Y_2|X_T) = P(X_T|Y_2) P(Y_2)$$

$$\vdots$$

$$P(Y_K|X_T) = P(X_T|Y_K) P(Y_K)$$

} ignoring denominator because, we are compare each and everything, since deno X_T is same across all comparison, so ignored

$$X_T = \{X_1, X_2, X_3, \dots, X_n\}$$

no. of i/p / features

$$Y_K = \{Y_1, Y_2, Y_3, \dots, Y_K\}$$

multi-classes o/p

$$P(Y_K|X_T) = \frac{P(X_T|Y_K) P(Y_K)}{P(X_1 \cap X_2 \cap X_3 \cap \dots \cap X_n | Y_K) P(Y_K)}$$

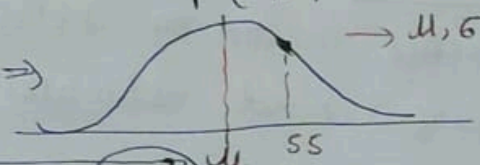
During Testing:-

- A new unseen data point is given to the model
- for each class, the model starts with prior prob $P(Y)$
- It multiplies the prior with all conditional prob of the feature $P(Y) \times \prod_{i=1}^n P(x_i | Y)$ formula
- This gives posterior score for that class (1 class)
- The same calculation is done for all classes (all classes)
- The class with the highest prior prob is chosen as Predict
- In practice, logarithms are used to avoid numerical underflow

How Naive Bayes handles numerical data:-

age	married	Each day	new data $\Rightarrow SS \Rightarrow ? \{Y, N\}$
27	Y μ, σ	$P(Y SS) = P(SS Y) P(Y)$ $P(N SS) = P(SS N) P(N)$	$\{Y, N\}$
61	N μ, σ		
52	Y μ, σ		
:	:		

i) Gaussian dist $\Rightarrow$



if SS near to μ , high
if SS is far from μ , low

$$f(x) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{1}{2} \left(\frac{x - \mu}{\sigma} \right)^2}$$

- Naive Bayes cannot count frequencies, it assumes a Gaussian (normal) distribution. This version is called Gaussian Naive Bayes
- During training, for each feature and each class, it calculates mean and variance
- During testing, the feature value is plugged into the Gaussian Probability density function. ($f(x)$)
- This gives $P(x_i | Y)$ for numerical features
- all feature probabilities are multiplied with class prior
- the class with highest posterior probability is predicted

Naive Bayes Formula

$$P(y | x_1, x_2, x_3, \dots, x_n) \propto \boxed{P(y) \prod_{i=1}^n P(x_i | y)}$$

from Bayes Theorem

$$P(y | x_1, x_2, x_3, \dots, x_n) = \frac{P(y) \overset{\text{multiplication}}{\left(\prod_{i=1}^n P(x_i | y) \right)}}{P(x_1, x_2, x_3, \dots, x_n)} \Rightarrow \frac{P(y) \times P(x_1 | y) \times P(x_2 | y) \times \dots \times P(x_n | y)}{P(x_1, x_2, x_3, \dots, x_n)}$$

→ $P(y) \Rightarrow$ prior

→ $P(x_i | y) \Rightarrow$ likelihood

→ denominator is same for all classes, so it's ignored during Comparison

What if data is not Gaussian:-

- Gaussian Naive Bayes assumes feature follow a normal distribution, if the data is not Gaussian, prob estimates may be inaccurate
- model performance can reduce, but still work reasonably well

why it can still work:-

- Naive Bayes ~~still~~ relies more on relative probs than exact values. Even imperfect prob estimates can give correct class ranking.

Solutions:-

- Apply log or power transformation to reduce skewness
- Discretize numerical features into bins & treat them as categorical (Categorical Naive Bayes)
- use kernel density Estimation instead of Gaussian dist
- switch to multinomial or Bernoulli Naive Bayes
(text, count data) binary features

when Naive is not ideal:-

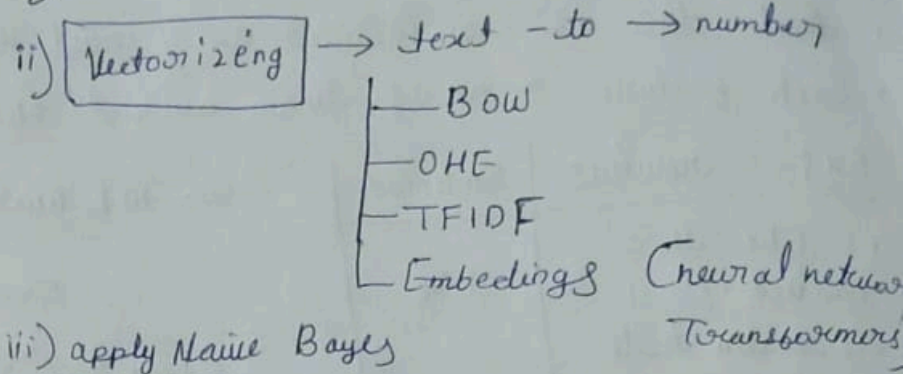
- feature distⁿ are highly irregular or multi-modal
- strong feature dependence exists
- High precision Probability is required

Naive Bayes On Text Data:-

Ex:- Sentiment Analysis

review	Sentiment
adhipurush worst movie even seen	-ve
Raasthana best movie, I liked very much	+ve

i) Text Processing



Step 1:- Text Preprocessing

- Convert text to lower case
- Remove Punctuation and special char
- Tokenize text into words
- Remove stopwords
- Apply stemming or lemmatization

Step 2:- Convert text to Numerical form

- Build vocabulary from the training data
- represent each document using Bag of words (Bow) or TF-IDF
- each feature represents a word & its count or frequency

Step 3:- Choose Naive Bayes Variant

- multinomial Naive is commonly used for text
- It models word count or frequencies, not continuous values

Step 4:- Training Phase

- Calculate Prior Prob $P(Y)$ of each class
- Count how many times each word appears in each class
- Calculate Conditional probability for each word $P(\text{word} | Y)$
- apply Laplace smoothing to avoid zero probability

Step 5:- Testing Phase

- Take a new unseen document
- Convert it to the same Bag of words / TF-IDF representation
- for each class compute $P(Y) \times \prod P(\text{word} | Y)$
- choose class with highest probability

1. Bag of words (Bow):

How it works

- Create a vocabulary from all training doc
- Each doc is represented as a count vector
- Each feature = no of time word appears

Ex:-

Reviews	Sentiment
I like the movie	+ve
I really hate the movie especially the actor	-ve

$\Rightarrow 10L \text{ words} \Rightarrow \frac{35000}{\text{vocabulary}} \text{ \& unique}$
 $\Downarrow$
 5000 most frequently used word
 $(\text{vocab}) \Rightarrow (35000, 5000)$ these becomes features

Vectorization

	hate	like	movie	actor
0	1	1	1	1
2	0	3	2	2
1	2	3	1	1

when new unseen text data

is given $\Rightarrow$ (adhipath is worst movie I ever seen, I hate the screenplay)

$\Rightarrow (0 \ 2 \ 1 \ \dots \ 5) \Rightarrow \text{vectorization}$
 $\downarrow$
 +ve $\quad \quad \quad$ -ve

$\rightarrow P(+ve | \text{hate} = 0, \text{like} = 2, \dots) \Rightarrow 0.23$
 $\rightarrow P(-ve | \text{hate} = 2, \text{like} = 1, \dots) \Rightarrow 0.37 \rightarrow +ve \text{ statement}$

Ex 2:- documents:

- D1 (spam): free money now
- D2 (ham): meeting now

$\rightarrow$ Vocabulary: [free, money, now, meeting]

Bow representation:-

document	free	money	now	meeting
D1	1	1	1	0
D2	0	0	1	01

apply Naive Bayes

$\rightarrow$ compute $P(\text{word} | \text{class})$
 $\rightarrow$ select word with high prob class

Pros:-

- Simple and fast
- works very well with multinomial

Cons

- common word like now get too much importance

TF-IDF with Naive Bayes

How it works

- TF (Term frequency) : how often a word appears in doc
- IDF (Inverse Document Frequency) : downweights common word
- final value $\Rightarrow$ $TF \times IDF$
- Rare but important word get higher weight

Same Example

- now appears in both doc $\rightarrow$ low IDF
- free & money appear only in spam $\rightarrow$ high IDF
- meeting appears only in ham $\rightarrow$ high IDF

doc	free	money	now	meeting
D1	high	high	low	0
D2	0	0	low	high

Naive Bayes TF-IDF values instead of scalar counts

Treats them as feature evidence

works well when word importance matters more than frequency.

Pros:-

- improves accuracy in text classification

Cons:-

- slightly breaks pure prob assumption
- less interpretable than B

One-Hot Encoding (OHE)

How it works

- Each unique word/category becomes a separate binary feature
- Value is 1 if present, 0 if absent

Ex:- Vocabulary : [free, money, meeting]

document : "free meeting"

OHE vector $\Rightarrow$ [1 0 1]

} frequency does matter only exists